

Distributed Analysis and Predictive Modelling of Short-Term Rentals: A PySpark Pipeline Over Porto, Lisbon, Madrid and Barcelona Airbnb Data

*Big Data Engineering (EGD), MECD, FEUP

Daniel Dória
MECD

Faculty of Engineering, UP
Porto, Portugal
up202108808@up.pt

Diego Montoya
MECD

Faculty of Engineering, UP
Porto, Portugal
up202502851@up.pt

Jorge Gamonal
MECD

Faculty of Engineering, UP
Porto, Portugal
up202602660@up.pt

José Evans
MECD

Faculty of Engineering, UP
Porto, Portugal
up202108818@up.pt

Miguel Santos
MECD

Faculty of Engineering, UP
Porto, Portugal
up202008450@up.pt

Abstract—In this work, we present a complete pipeline that covers the entire process of analytical exploration and predictive modeling of Inside Airbnb data from four cities (Porto, Lisbon, Madrid, and Barcelona), running on Apache Spark. After data ingestion and cleaning, we run eleven analytical queries and train three machine-learning models using an MLlib pipeline. We highlight two main findings. Firstly, our models proved highly capable of leveraging the extracted features to produce accurate predictions. Our price-prediction tool was remarkably accurate, achieving $R^2 = 0.72$, and generally outputting predictions within 78€ of the actual price. Beyond just price, we successfully trained algorithms to reliably spot which properties will be consistently booked up over the long term, as well as predict the busiest, highest-demand months of the year. Second, a scalability study on Google Dataproc, which benchmarks five workloads across one, two, and four worker nodes, shows that join-heavy tasks achieve a speedup of 1.98 when scaling from one to two nodes. However, we observe diminishing returns beyond this point, as fixed overheads become the bottleneck at this data scale.

Index Terms—Apache Spark, MLlib, distributed computing, Airbnb, price prediction, occupancy forecasting, big data engineering

I. INTRODUCTION

The short-term rental sector has become a major component of urban tourism, with platforms such as Airbnb listing millions of properties worldwide. The Inside Airbnb project [1] publicly scrapes per-city snapshots of listings, availability calendars and reviews, producing a large, heterogeneous dataset that has motivated both academic research and policy debate.

Although we restrict the data to four European cities, the data is not trivial. Our combined snapshot contains 25.8M calendar rows and 4.90M reviews, a volume that simple workflows struggle to process. This is the motivation be-

hind employing a Spark-based distributed pipeline. This work makes four contributions:

- 1) A reproducible PySpark pipeline (ingest $\rightarrow$ clean $\rightarrow$ query $\rightarrow$ model) operating over Porto, Lisbon, Madrid and Barcelona;
- 2) Eleven analytical queries, encompassing price, host-concentration and seasonality centered analyses;
- 3) Three machine-learning models on a shared MLlib feature pipeline: price regression, annual occupancy classification, and a month-aware occupancy classifier trained on the calendar $\times$ listings join;
- 4) A scalability study on Google Dataproc, benchmarking five workloads using 1, 2 and 4 worker nodes.

II. RELATED WORK

Hedonic-pricing models are the long-standing baseline for short-term-rental valuation. Wang and Nicolau [2] fit a hedonic model on a 33-city Airbnb panel and showed that location, host attributes, room type and amenities together account for roughly 40% of price variance. Chen and Xie [3] applied a similar framework to Austin listings, isolating a significant effect for professionally managed multi-listing hosts. The host's listing count is likewise among the most important price features in our model (Section V-E). Machine-learning approaches achieved improvements in price prediction models. Kalehbasti et al. [4] combined gradient boosting with review-sentiment analysis on New York City data and report $R^2 \approx 0.69$ on log-price. Tang and Sangani [5] earlier achieved $R^2 \approx 0.65$ on San Francisco data using support-vector regression over a comparable feature set. Our gradient-boosted model reaches $R^2_{\log} = 0.715$ across the four Iberian cities using only the structural features that Inside Airbnb

publishes (no listing text or review sentiment), suggesting that careful encoding of high-cardinality categorical variables can partially substitute for textual features. Hotel demand forecasting has a long story [6], [7], with recent work [8] employing deep-learning models to predict tourism related occupancy rates. The Airbnb-specific literature on occupancy prediction is comparatively sparse, with most studies focusing on price or growth. Our month level classifier targets this gap, using the daily availability calendar as the demand signal. We employ standard the cyclic month encoding to preserve the adjacency of December and January. Regarding the tools chosen for this project, Apache Spark [9] is the foundational engine for our distributed processing. We make extensive use of MLlib’s Pipeline API [10]. For the high-cardinality neighbourhood field we apply Micci-Barreca’s target encoding [11], computing the per-neighbourhood mean log-price on the training split only to prevent leakage. The Iberian context is documented by Gutiérrez et al. [12] (Barcelona) and Cocola-Gant [13] (Lisbon tourism gentrification), both of which highlight the spatial concentration patterns we observe. For the tree ensembles we apply Random Forests

III. DATA AND PREPARATION

A. Data sources

We use Inside Airbnb snapshots for each city. Two CSV variants are joined on listing identifier: a *summary* file (18 columns; clean structural fields, integer prices) and a *detailed* file (79 columns; richer structural, host and review attributes, with a dollar-formatted price). The principal fields used throughout this study are catalogued in Table I. In the summary file the nightly price is missing for a sizeable share of rows (from $\approx 11\%$ in Porto to $\approx 24\%$ in Madrid, about 18% overall), so the cleaning stage merges the two price sources, preferring the detailed value where available, to maximise coverage.

B. Cleaning and statistics

After casting the dollar-formatted price to a numeric type, dropping rows with null or zero price, deduplicating by listing identifier, and joining the detailed file, we obtain 68,521 listings. Table II summarises the cleaned data per city.

TABLE II
CLEANED DATASET STATISTICS PER CITY

City	Listings	Median €	Calendar	Reviews
Porto	13,189	90	5.43 M	1.00 M
Lisbon	21,466	109	8.98 M	1.68 M
Madrid	18,953	110	9.13 M	1.27 M
Barcelona	14,913	143	2.30 M	0.95 M
Total	68,521	—	25.84 M	4.90 M

C. Storage and partitioning

All processed tables are written as Snappy-compressed Parquet, partitioned by city. Partitioning enables partition pruning during single-city queries and parallel reads across partitions

for cross-city operations, and it is the physical layout the scalability study exercises (Section VI).

D. Exploratory findings

Three descriptive patterns emerge from initial profiling. First, the median nightly price is highest in Barcelona (€143), comparable in Lisbon (€109) and Madrid (€110), and significantly lower in Porto (€90). Second, room-type composition is similar across the four cities, with entire homes or apartments dominating everywhere, from 71% of listings in Barcelona to 84% in Porto. Third, host concentration is high throughout: hosts with more than one listing hold between 75% (Lisbon) and 81% (Barcelona) of available properties, consistent with the professionalisation effect reported by Wang and Nicolau [2].

IV. ANALYTICAL QUERIES

Each query is implemented as a self-contained module exposing a uniform `run(spark) → DataFrame` interface and registered in a central catalogue. A single driver job runs any query by identifier and transforms its result to Parquet, decoupling computation from the downstream consumers (analysis notebooks, an interactive Streamlit dashboard, and this report). Table III lists all eleven queries. They span the three dataset types (listings, calendar, reviews) and cover both descriptive analytics (price distributions, host concentration) and joins of meaningful big-data scale: Q2 in particular joins the 25.8 M-row calendar with the 68.5 K-row listings table.

TABLE III
ANALYTICAL QUERY CATALOGUE

ID	Question / output
Q1	Top and bottom 10 neighbourhoods by median price per city
Q2	Monthly availability and occupancy (calendar $\times$ listings)
Q3	Average availability and occupancy by room type $\times$ city
Q4	Multi-listing vs. single-listing host comparison
Q5	Host concentration (Pareto curve of listings per host)
Q6	Pearson correlation: reviews/month $\times$ price per neighbourhood
Q7	Top 10 neighbourhoods by estimated annual revenue
Q8	Room-type market share per city
Q9	Monthly review trend and year-over-year growth
Q10	Price-outlier share (€100/€250/€500) per room type
Q11	District/municipality comparison (higher-level grouping)

We emphasize the three queries that illustrate interesting patterns discussed in the related work section. Q5 confirms a heavy Pareto distribution of inventory (Fig. 1), where the top 10% of hosts holds essentially half of all listings in Porto and Lisbon (49% in both), rising to 56% in Madrid and 59% in Barcelona. In every city the curve sits well above the equal-share diagonal. Q2 reveals heterogeneous monthly demand (Fig. 2). The three coastal-tourism cities peak in June–July, with a June–August occupancy lift of +34% above the annual mean in Barcelona, +39% in Lisbon and +61% in Porto, whereas inland Madrid carries only a mild summer lift (+7%)

TABLE I
DATA DICTIONARY: PRINCIPAL FIELDS USED IN THE ANALYSIS AND MODELS

Field	Description
<i>Structural and capacity</i>	
accommodates	Guest capacity (number of guests the listing sleeps)
bedrooms, beds, bathrooms	Room, bed and bathroom counts
property_type	Dwelling type (entire apartment, house, etc.)
room_type	Entire home / private room / hotel room / shared room
amenity_count	Number of amenities listed (derived from the amenities array)
minimum_nights	Minimum nights required per booking
<i>Location</i>	
latitude, longitude	Geographic coordinates
neighbourhood	Cleansed neighbourhood name (483 distinct across the four cities)
neighbourhood_group	Higher-level grouping: district (Spanish cities) or municipality (Portuguese cities)
<i>Host</i>	
calculated_host_listings_count	Number of active listings managed by the host
host_is_superhost	Airbnb “superhost” status flag
<i>Reviews and activity</i>	
number_of_reviews, number_of_reviews_ltm	Lifetime and last-12-month review counts
reviews_per_month	Average monthly review rate
review_scores_*	Guest rating sub-scores (overall, cleanliness, location, value)
<i>Price and availability</i>	
price	Nightly price in € (source of the regression target)
availability_365	Bookable days over the next year
occupancy_rate	$1 - \text{availability_365}/365$, used as an annual demand proxy
<i>Engineered for modelling (Section ??)</i>	
neighbourhood_mean_log_price	Target encoding: mean training log-price of the neighbourhood
log_reviews, log_host_listings	Log-transformed skewed counts
recent_activity_ratio	Share of a listing’s reviews received in the last 12 months
min_nights_cat	Bucketed minimum-nights category

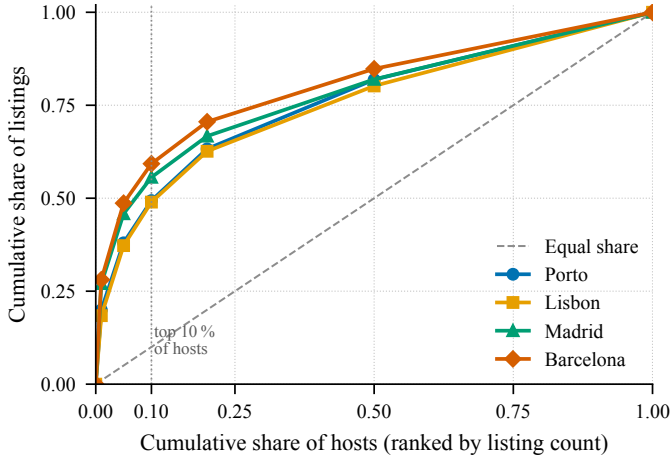


Fig. 1. Host-concentration Pareto curves per city.

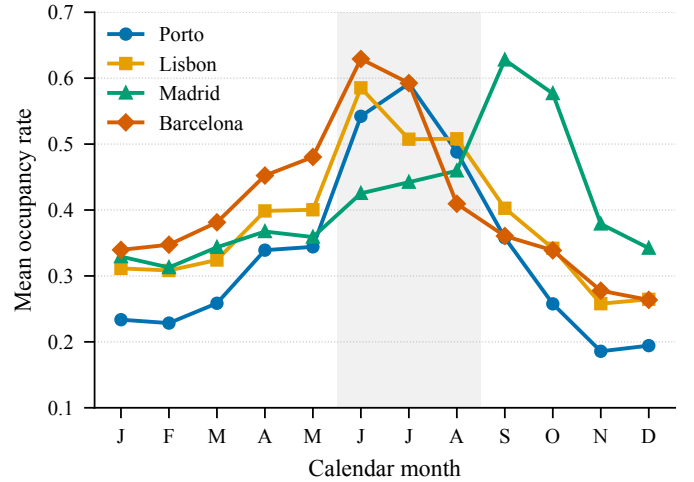


Fig. 2. Monthly occupancy rate per city, averaged across the available calendar window.

and instead reaches its highest occupancy in September–October, consistent with a year-round business and city-break profile rather than a leisure-season one. **Q11** shows that in the two Spanish cities a single central district dominates its city’s inventory. Madrid’s Centro (≈ 8.1 K listings) and Barcelona’s Eixample (≈ 5.4 K), confirming the spatial concentration documented for Mediterranean tourism arkets [12].

V. PREDICTIVE MODELING METHODOLOGY

A. Shared ML pipeline

All three models share a single MLib pipeline, parameterized by the target column and terminated by the model’s estimator. The four preprocessing stages are:

- 1) **Feature engineering** — derive seven columns: the annual occupancy rate, log-transformed review and

host-listing counts, a recent-activity ratio, a bucketed minimum-nights category, and the two-column neighbourhood target encoding (Table I);

- 2) **Selection and imputation** — project to the model columns and impute missing structural fields (bedrooms, ratings, etc.) with proper defaults;
- 3) **Categorical encoding** — transform categorical variables through the sequential application of `StringIndexer` and `OneHotEncoder`;
- 4) **Feature assembly and scaling** — aggregate all features into a unified vector representation via `VectorAssembler`, followed by `StandardScaler`. Scaling is applied without mean centering to establish comparable feature magnitudes for linear regression while accommodating the innate scale-invariance of tree-based models.

Target encoding for high-cardinality categoricals: The neighbourhood field has 483 distinct values across the four cities — too many for efficient one-hot encoding. Following Micci-Barreca [11], we replace it with the mean $\log(1 + \text{price})$ of training listings in that neighbourhood. These statistics are computed on the training split only, preventing target leakage.

B. Feature Engineering and Model Architectures

Price regression: The target is $y = \log(1 + \text{price})$. A dynamic cap at the 99th percentile ($p_{99} = \text{€}1003$) removes high-end outliers that disproportionately inflate the squared-error loss. Three estimators are trained on an 80/20 split (seed 42):

- **Ridge Linear Regression** [14] — L_2 regularisation, $\lambda = 0.01$;
- **Random Forest** [15] — 100 trees, depth 8, minimum 10 instances per node;
- **Gradient-Boosted Trees** [16] — 200 iterations, depth 6, step size 0.05, subsample rate 0.8.

Annual occupancy classifier: A listing is labelled *high-occupancy* when its booked fraction, $1 - \text{availability}_{365}/365$, exceeds 0.70. To prevent leakage, all availability-derived fields (including the occupancy rate and the booked-night and revenue quantities computed from them) are excluded from the feature set. The base rate on the training data is 21%, so the classifiers use inverse-frequency class weighting. We evaluated both logistic regression and a random forest (150 trees, depth 12).

Month-aware occupancy classifier: The third model joins the daily calendar with the listings table to construct a (listing, year, month) panel of approximately 684,000 observations. A monthly observation is retained only where at least 20 calendar days are available, and the target marks months whose occupancy rate exceeds 0.50, a threshold chosen so the base rate is balanced at 37% rather than the much lower long-run annual base rate. The model adds three temporal features to the shared pipeline: the month index $m \in \{1, \dots, 12\}$ and its cyclic pair $\sin(2\pi m/12)$, $\cos(2\pi m/12)$. The trigonometric encoding makes the model treat December and January as

adjacent, which an ordinal index would not. A random forest estimator is then trained on this expanded feature set.

C. Results

Table IV reports both log-scale and back-transformed Euro-scale metrics. The gap between $R_{\log}^2$ and $R_{\text{€}}^2$ reflects the outlier-sensitivity of the squared error after exponentiation, where small errors in log-space become large absolute errors at the top of the price distribution.

TABLE IV
PRICE-REGRESSION RESULTS ON THE HELD-OUT TEST SET ($n = 13,341$)

Model	Log-scale			Euro-scale		
	RMSE	MAE	R^2	RMSE	MAE	R^2
LR (ridge)	0.430	0.319	0.617	87.3	46.2	0.473
RF	0.408	0.302	0.655	86.4	44.6	0.484
GBT	0.370	0.267	0.715	77.9	39.7	0.581

D. Occupancy classification performance

For the annual occupancy classifier, the random forest achieved $\text{AUC} = 0.73$ and $F_1 = 0.74$; logistic regression registered lower scores at $\text{AUC} = 0.64$. This performance gap reflects the importance of non-linear interactions between location, room type and host professionalism that the tree ensemble captures. The month-aware occupancy classifier random forest achieved $\text{AUC} = 0.77$ on the held-out 136,773-row test set. This is not directly comparable with the annual classifier’s 0.73, since the two models target different occupancy thresholds (0.50 vs 0.70) on different units (listing-month panel vs listing), but it demonstrates that high-demand months are highly learnable from a listing’s structural features combined with a cyclic month encoding.

E. Feature importance

Table V lists the ten most important features for the best-performing GBT price model, aggregating the one-hot expansions of categorical features back to their parent column. The ranking aligns with the hedonic-pricing literature [2], [3]: capacity and structural attributes (guest capacity, property and room type, amenity count), location (the neighbourhood target encoding, longitude, the district grouping) and host professionalism (the host’s listing count) dominate. The superhost flag and the log-transformed host/review counts carried zero importance, and instant-bookability was negligible ($< 0.2\%$).

TABLE V
TOP-10 FEATURES BY IMPORTANCE, GBT PRICE MODEL

Feature	Importance
accommodates	0.106
property_type	0.094
neighbourhood_mean_log_price	0.070
calculated_host_listings_count	0.065
minimum_nights	0.051
room_type	0.047
longitude	0.045
neighbourhood_group	0.041
amenity_count	0.039
review_scores_cleanliness	0.037

VI. PERFORMANCE AND SCALABILITY

To characterise how the pipeline scales with cluster size, we benchmark five representative workloads on Google Dataproc across 1, 2 and 4 worker nodes.

A. Workloads

The five workloads span the two dominant cost profiles in the pipeline, distributed shuffles and iterative ML training:

- **Q2 seasonality** — joins the 26 M-row calendar with listings and aggregates to (city, year, month); shuffle-bound but emits only 53 rows;
- **Calendar revenue** — the same join plus a per-month neighbourhood revenue aggregation and a windowed ranking;
- **Review demand** — joins the 4.95 M-row reviews table with listings and aggregates monthly review volume by price band, room type and city;
- **Full pipeline** — a three-way calendar $\times$ listings $\times$ reviews join, the heaviest workload by wall-clock time;
- **RF price train** — fits a representative random-forest price regressor (50 trees, depth 8) over the listings table.

B. Methodology and configuration

Runs execute on Dataproc clusters of 1, 2 and 4 `n2-standard-4` workers (4 vCPUs / 16 GB each; image 2.2-debian12, Spark 3.5), giving 4, 8 and 16 executor cores respectively. The benchmark reads a Google Cloud Storage copy of the four-city corpus in the same Snappy-Parquet layout (69,647 listings, 26.20 M calendar rows and 4.95 M reviews, within $\approx 2\%$ of Table II; the small difference reflects a slightly earlier scrape). Each workload is timed three times per configuration with the Spark cache cleared between runs, and we report the median wall-clock time. The first run of every triple is a consistent cold-start outlier (roughly 2–3.5 $\times$ slower) caused by executor JVM warm-up, broadcast and GCS metadata latency; being the slowest of the three, it is never selected as the median. Speedup is $S(n) = T(1)/T(n)$ and parallel efficiency $E(n) = S(n)/n$, both relative to the single-worker baseline.

C. Results

Table VI and Fig. 3 report the measurements.

TABLE VI
DATAPROC WALL-CLOCK SECONDS (MEDIAN OF 3 RUNS) AND SPEEDUP RELATIVE TO THE 1-WORKER BASELINE

Workload	Wall-clock (s)			Speedup	
	1w	2w	4w	$S(2)$	$S(4)$
Q2 seasonality	22.4	22.5	17.1	1.00	1.31
Calendar revenue	26.5	16.6	17.4	1.59	1.53
Review demand	36.9	18.7	15.5	1.98	2.39
Full pipeline	40.2	22.9	22.7	1.75	1.77
RF price train	60.5	39.7	40.6	1.53	1.49

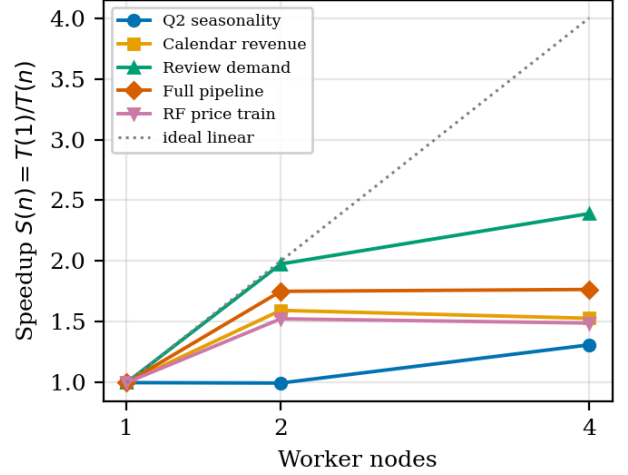


Fig. 3. Speedup versus worker-node count on Dataproc. The heaviest shuffle (review demand) scales closest to linear; lighter workloads plateau after two workers.

We identify three main findings. First, scaling from one to two workers delivers most of the benefit: heavy joins gain $S(2) \approx 1.6$ –2.0, with the heaviest workload (review demand, a 4.95 M-row join) closest to linear at $S(2) = 1.98$. Second, the step from two to four workers yields little further speedup: most workloads plateau ($S(4) \approx S(2)$) and parallel efficiency drops to $E(4) \approx 0.33$ –0.60. At these data volumes (tens of millions of rows; a few hundred MB of Parquet) a 16-core cluster is over-provisioned, so fixed per-stage overheads (task scheduling, shuffle setup, broadcast of the listings table, GCS I/O latency) dominate the shrinking per-core work. Third, the benefit tracks workload size: review demand and the full three-way join scale furthest, whereas Q2 seasonality — which collapses 26 M rows into a 53-row result — barely benefits ($S(4) = 1.31$) because its final aggregation is cheap and latency-bound. RF training is compute-bound and gains from the first doubling ($S(2) = 1.53$) but not beyond, as the ≈ 55 K-row training split is small relative to the broadcast and coordination cost of the growing ensemble.

The engineering lesson is consistent with distributed-computing theory: distribution pays off in proportion to shuffle and compute volume, and for this corpus two `n2-standard-4` workers already saturate the available parallelism. Sustained linear scaling to four nodes and beyond

would require larger inputs (more cities or per-night granularity) or heavier models.

VII. DISCUSSION

Cross-city structure. The clearest economic signal is a price gradient that tracks tourism intensity (Barcelona > Madrid $\approx$ Lisbon > Porto) and is only partly captured by the structural features in our model: the neighbourhood target encoding alone is the third most important GBT feature (Table V). Host concentration, by contrast, is strikingly uniform (75–81% of inventory held by multi-listing operators), reinforcing the professionalisation thesis. Seasonality is the dimension that separates the markets most sharply, with a pronounced June–July peak in the coastal cities versus a delayed September–October peak in inland Madrid (Fig. 2); both patterns are signals that the month-aware model exploits.

Price model in context. The GBT log- R^2 of 0.715 marginally exceeds the $R^2 \approx 0.69$ of Kalehbasti et al. [4] on New York and the $R^2 \approx 0.65$ of Tang and Sangani [5] on San Francisco, despite using only published structural features (no review text or sentiment). We attribute this partly to the target-encoded neighbourhood feature, which captures fine-grained location signal that one-hot encoding would dilute across 483 sparse columns. The Euro-scale RMSE of €78, against per-city medians in the €90–€143 range (Table II), reflects the heavy right tail of the price distribution rather than systematic bias, since the log-scale fit is strong.

Monthly demand. The month-aware classifier reaches AUC = 0.77 on a balanced (37%) base rate; while the comparison with the annual model is informal (the two operate on different targets), the result confirms that monthly demand is learnable from listing structure combined with a cyclic month encoding.

Limitations. The snapshots contain nulls in the summary-format price (mitigated by the join done with the detailed file) and lack a usable adjusted price in the calendar, which makes true per-night dynamic-pricing analysis impossible. The Barcelona snapshot also carries a shorter forward calendar horizon than the other three cities (about 154 days per listing versus a full year); cross-city seasonality is therefore reported as occupancy rates rather than raw day counts to remain comparable.

VIII. REPRODUCIBILITY

The pipeline is organised as a single repository in which the ingest, clean, query and training stages are exposed as command-line jobs and the generated artifacts (query results, trained model pipelines, feature importances and benchmark logs) are released alongside the code. All train/test splits use a fixed random seed (42). The software stack is Python 3.11 with PySpark 3.5 and MLlib; the cloud benchmark uses Dataproc image 2.2-debian12 and is driven by a standalone PySpark script together with the cluster-provisioning scripts in the repository. The four-city Inside Airbnb snapshots are public [1].

IX. CONCLUSION

We built and evaluated a complete distributed pipeline for short-term-rental analytics across Porto, Lisbon, Madrid and Barcelona: 68,521 listings, 25.8M calendar rows and 4.90M reviews ingested into Parquet, processed by eleven analytical queries, and modelled by three MLlib pipelines. The best price regressor (GBT) attains $R^2_{\log} = 0.72$, RMSE = €78; the annual occupancy classifier reaches AUC = 0.73, and a month-aware variant rises to AUC = 0.77 on a 684K-row calendar panel. The Dataproc scalability study confirmed that the heavier shuffle workloads scale near-linearly to two worker nodes but plateau thereafter, the cluster being over-provisioned for this data volume.

Future work includes (i) integrating streaming calendar updates to enable dynamic price recommendation, (ii) extending the pipeline to further European cities for cross-market generalisation, and (iii) replacing manual hyperparameter choices with Spark CrossValidator grid search on the Dataproc cluster.

REFERENCES

- [1] M. Cox, “Inside Airbnb: Adding data to the debate,” <http://insideairbnb.com>, 2024.
- [2] D. Wang and J. L. Nicolau, “Price determinants of sharing economy based accommodation rental: A study of listings from 33 cities on Airbnb.com,” *International Journal of Hospitality Management*, vol. 62, pp. 120–131, 2017.
- [3] Y. Chen and K. Xie, “Consumer valuation of Airbnb listings: a hedonic pricing approach,” *International Journal of Contemporary Hospitality Management*, vol. 29, no. 9, pp. 2405–2424, 2017.
- [4] P. R. Kalehbasti, L. Nikolenko, and H. Rezaei, “Airbnb price prediction using machine learning and sentiment analysis,” *arXiv preprint arXiv:1907.12665*, 2019.
- [5] E. Tang and K. Sangani, “Neighborhood and price prediction for San Francisco Airbnb listings,” Stanford CS229 Final Project, 2015.
- [6] Y. Yang, B. Pan, and H. Song, “Predicting hotel demand using destination marketing organization’s web traffic data,” *Journal of Travel Research*, vol. 53, no. 4, pp. 433–447, 2014.
- [7] B. Pan, D. C. Wu, and H. Song, “Forecasting hotel room demand using search engine data,” *Journal of Hospitality and Tourism Technology*, vol. 3, no. 3, pp. 196–210, 2012.
- [8] Y. Zhang, G. Li, B. Muskat, and R. Law, “Tourism demand forecasting: A decomposed deep learning approach,” *Journal of Travel Research*, vol. 60, pp. 981–997, 06 2020.
- [9] M. Zaharia, R. S. Xin, P. Wendell, T. Das, M. Armbrust, A. Dave, X. Meng, J. Rosen, S. Venkataraman, M. J. Franklin, A. Ghodsi, J. Gonzalez, S. Shenker, and I. Stoica, “Apache Spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [10] X. Meng, J. Bradley, B. Yavuz, E. Sparks, S. Venkataraman, D. Liu, J. Freeman, D. Tsai, M. Amde, S. Owen et al., “MLlib: Machine learning in Apache Spark,” *Journal of Machine Learning Research*, vol. 17, no. 34, pp. 1–7, 2016.
- [11] D. Micci-Barreca, “A preprocessing scheme for high-cardinality categorical attributes in classification and prediction problems,” *ACM SIGKDD Explorations Newsletter*, vol. 3, no. 1, pp. 27–32, 2001.
- [12] J. Gutiérrez, J. C. García-Palomares, G. Romanillos, and M. H. Salas-Olmedo, “The eruption of Airbnb in tourist cities: Comparing spatial patterns of hotels and peer-to-peer accommodation in Barcelona,” *Tourism Management*, vol. 62, pp. 278–291, 2017.
- [13] A. Cocola-Gant, “Tourism gentrification,” in *Handbook of Gentrification Studies*. Edward Elgar Publishing, 2018.
- [14] A. E. Hoerl and R. W. Kennard, “Ridge regression: Biased estimation for nonorthogonal problems,” *Technometrics*, vol. 12, no. 1, pp. 55–67, 1970.
- [15] L. Breiman, “Random forests,” *Machine learning*, vol. 45, no. 1, pp. 5–32, 2001.

- [16] J. H. Friedman, “Greedy function approximation: A gradient boosting machine,” *Annals of Statistics*, vol. 29, no. 5, pp. 1189–1232, 2001.